PRAKTIKUM MODUL 6



Disusun Oleh:

Akmallutfi Afza Gusty

707012500064

SIKC 49 - 04

Mata Kuliah:

Pengembangan Aplikasi Berbasis Web

Dosen Pengampu:

Diash Firdaus

1. Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep authentication, session, hashing password, dan middleware pada Laravel.

2. Menginstal dan menggunakan Laravel Breeze untuk membuat fitur register, login, dan logout secara otomatis.

3. Memahami alur kerja register, login, logout, dan middleware auth di balik layar.

4. Mampu mengimplementasikan database seeding dengan Faker untuk membuat akun pengguna (user) dummy.

5. Mampu melindungi halaman aplikasi agar hanya dapat diakses oleh pengguna yang sudah login.

2. Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

Perangkat Keras:

- Komputer atau laptop dengan sistem operasi Windows 11 Home Single Language (versi 10.0.26200), prosesor Intel Core Ultra 7 255HX, RAM 16 GB, dan ruang penyimpanan kosong sekitar 171 GB pada drive C:.

Perangkat Lunak:

- Laragon sebagai server lokal (Apache dan MySQL 8.4.3), termasuk phpMyAdmin 6.0.0 untuk mengelola database.
- Composer 2.10.3 sebagai dependency manager PHP.
- PHP 8.5.10 (CLI) yang dipakai project ini.
- Laravel Framework 13.37.0 dan paket laravel/breeze v2.4.2.
- Node.js v24.14.0, NPM 11.9.0, dan Vite 8.3.4 untuk meng-compile asset tampilan Breeze.
- Visual Studio Code sebagai teks editor.
- Google Chrome sebagai browser untuk menguji aplikasi.

3. Langkah-Langkah Praktikum

3.1. Instalasi Laravel Breeze dan Konfigurasi Project

Praktikum ini melanjutkan project Laravel dari modul sebelumnya, sehingga struktur aplikasi sudah tersedia dan yang perlu ditambahkan hanya fitur authentication-nya. Langkah pertama adalah memasang paket Laravel Breeze ke dalam project dengan perintah composer require laravel/breeze --dev. Paket tersebut terunduh ke folder vendor, kemudian pemasangan

dilanjutkan dengan perintah php artisan breeze:install blade. Perintah ini menimpa routes/web.php, membuat route authentication pada routes/auth.php, menambahkan controller ke folder app/Http/Controllers/Auth, dan menyiapkan view login serta register di resources/views/auth.

Karena tampilan Breeze dibuat dengan Tailwind CSS dan di-compile memakai Vite, saya menjalankan npm install satu kali untuk mengunduh paket JavaScript, lalu npm run dev untuk meng-compile asset sambil memantau perubahan file. Terminal npm run dev harus tetap terbuka selama aplikasi diuji, dan php artisan serve dijalankan pada terminal terpisah.

```
> npm run dev

> dev
> vite

Port 5173 is in use, trying another one...

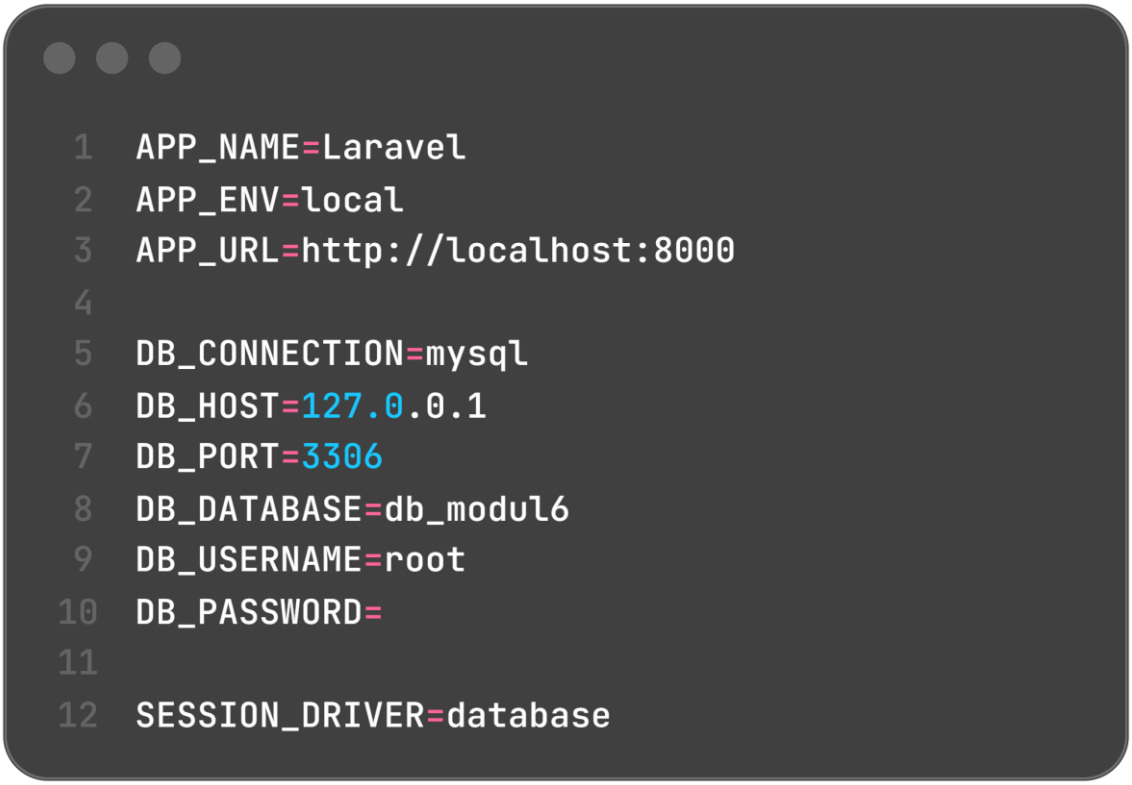
  VITE v8.3.4  ready in 148 ms

  Local:   http://localhost:5174/
  Network: use --host to expose

  LARAVEL v13.37.0  plugin v3.2.0

  APP_URL: http://localhost:8000
```

Database project ini memakai MySQL dengan nama db_modul6, mengikuti konfigurasi pada modul sebelumnya. Bagian .env yang berkaitan dengan database dan session adalah sebagai berikut.



```
APP_NAME=Laravel
APP_ENV=local
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_modul6
DB_USERNAME=root
DB_PASSWORD=

SESSION_DRIVER=database
```

Setelah koneksi database benar, migration dijalankan dengan php artisan migrate:fresh agar tabel users, password_reset_tokens, dan sessions terbentuk dari awal pada database yang bersih.

```
> php artisan migrate:fresh

  Dropping all tables ..................................... 38.06ms DONE

  INFO  Preparing database.

  Creating migration table ................................ 9.16ms DONE

  INFO  Running migrations.

  0001_01_01_000000_create_users_table ..................... 35.41ms DONE
  0001_01_01_000001_create_cache_table ..................... 20.64ms DONE
  0001_01_01_000002_create_jobs_table ...................... 33.70ms DONE

> php artisan migrate:status

  Migration name ........................................... Batch / Status
  0001_01_01_000000_create_users_table ..................... [1] Ran
  0001_01_01_000001_create_cache_table ..................... [1] Ran
  0001_01_01_000002_create_jobs_table ...................... [1] Ran
```

3.2. Konfigurasi Route Authentication

Breeze mengubah isi routes/web.php dan menambahkan satu baris require __DIR__.'/auth.php'; sehingga route authentication dipisahkan ke file tersendiri. Isi routes/web.php menjadi seperti berikut.

```php
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

Seluruh route register, login, logout, lupa password, dan verifikasi email berada di routes/auth.php. Route tersebut dibagi menjadi dua kelompok middleware, yaitu guest untuk pengguna yang belum login dan auth untuk pengguna yang sudah login.

```php
<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\ConfirmablePasswordController;
use App\Http\Controllers\Auth\EmailVerificationNotificationController;
use App\Http\Controllers\Auth\EmailVerificationPromptController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth\PasswordController;
use App\Http\Controllers\Auth\PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);

    Route::get('forgot-password', [PasswordResetLinkController::class, 'create'])
        ->name('password.request');

    Route::post('forgot-password', [PasswordResetLinkController::class, 'store'])
        ->name('password.email');

    Route::get('reset-password/{token}', [NewPasswordController::class, 'create'])
        ->name('password.reset');

    Route::post('reset-password', [NewPasswordController::class, 'store'])
        ->name('password.store');
});

Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');

    Route::get('verify-email/{id}/{hash}', VerifyEmailController::class)
        ->middleware(['signed', 'throttle:6,1'])
        ->name('verification.verify');

    Route::post('email/verification-notification',
[EmailVerificationNotificationController::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');

    Route::get('confirm-password', [ConfirmablePasswordController::class, 'show'])
        ->name('password.confirm');

    Route::post('confirm-password', [ConfirmablePasswordController::class, 'store']);

    Route::put('password', [PasswordController::class, 'update'])-
>name('password.update');

    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});
```

- Route::get('/', ...) menampilkan halaman welcome yang dipakai sebagai halaman awal aplikasi.
- Route::get('/dashboard', ...) menampilkan view dashboard dan dilindungi ->middleware(['auth', 'verified']).

- Route::middleware('guest') pada routes/auth.php menangani alamat /register, /login, /forgot-password, dan /reset-password, sehingga halaman tersebut tidak dapat dibuka oleh pengguna yang sudah login.
- Route::middleware('auth') menangani /logout, /profile, /confirm-password, dan route verifikasi email, sehingga hanya bisa diakses setelah login.

3.3. Controller Authentication

Proses register ditangani RegisteredUserController. Method create() hanya menampilkan form, sedangkan store() memvalidasi input, menyimpan user baru dengan password yang sudah di-hash, memicu event Registered, lalu langsung melakukan login dan mengarahkan pengguna ke dashboard.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Handle an incoming registration request.
     *
     * @throws ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect(route('dashboard', absolute: false));
    }
}
```

- Validasi nama wajib diisi dengan panjang maksimal 255 karakter, email harus berformat email, unik, dan disimpan dalam huruf kecil, serta password wajib dikonfirmasi dan mengikuti aturan bawaan Rules\Password::defaults().

- User::create([...]) menyimpan data akun ke tabel users melalui model User, dan Hash::make($request->password) mengubah password menjadi hash sebelum disimpan.
- Auth::login($user) membuat session login untuk pengguna yang baru mendaftar, sehingga ia tidak perlu login ulang.

Proses login dan logout ditangani AuthenticatedSessionController. Method store() memanggil authenticate() pada LoginRequest, meregenerasi session, lalu mengarahkan pengguna ke halaman yang dituju. Method destroy() melakukan logout, menghapus session, dan membuat ulang CSRF token.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    /**
     * Display the login view.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard', absolute: false));
    }

    /**
     * Destroy an authenticated session.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

Pemeriksaan email dan password sebenarnya terjadi di LoginRequest. Method authenticate() memanggil Auth::attempt() dengan email dan password dari form, dan apabila gagal maka permintaan dicatat oleh RateLimiter. Batas percobaan login yang gagal adalah 5 kali, setelah

itu pengguna harus menunggu sebelum dapat mencoba lagi. File LoginRequest.php saya tampilkan dalam dua bagian karena cukup panjang.

```php
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Auth\Events\Lockout;
use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;

class LoginRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }

    /**
     * Attempt to authenticate the request's credentials.
     *
     * @throws ValidationException
     */
    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();

        if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember')))
{
            RateLimiter::hit($this->throttleKey());

            throw ValidationException::withMessages([
                'email' => trans('auth.failed'),
            ]);
```

```php
// ... lanjutan class LoginRequest
        }

        RateLimiter::clear($this->throttleKey());
    }

    /**
     * Ensure the login request is not rate limited.
     *
     * @throws ValidationException
     */
    public function ensureIsNotRateLimited(): void
    {
        if (! RateLimiter::tooManyAttempts($this->throttleKey(), 5)) {
            return;
        }

        event(new Lockout($this));

        $seconds = RateLimiter::availableIn($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.throttle', [
                'seconds' => $seconds,
                'minutes' => ceil($seconds / 60),
            ]),
        ]);
    }

    /**
     * Get the rate limiting throttle key for the request.
     */
    public function throttleKey(): string
    {
        return Str::transliterate(Str::lower($this->string('email')).'|'.$this->ip());
    }
}
```

- Auth::attempt() mencocokkan email dengan baris pada tabel users, lalu memeriksa password memakai Hash::check() di dalamnya.
- ensureIsNotRateLimited() memakai RateLimiter::tooManyAttempts() dengan kunci gabungan email dan alamat IP, dan mengirim event Lockout jika batas terlampaui.
- $request->session()->regenerate() membuat ID session baru setelah login berhasil agar session fixation tidak terjadi.

3.4. Model dan Migration User

Model User sudah disediakan Laravel dan dipakai langsung pada praktikum ini. Model tersebut memakai trait HasFactory dan Notifiable, menentukan atribut fillable serta hidden melalui attribute PHP, dan meng-cast kolom email_verified_at sebagai datetime dan password sebagai hashed.

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Attributes\Hidden;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

#[Fillable(['name', 'email', 'password'])]
#[Hidden(['password', 'remember_token'])]
class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
}
```

Struktur tabel users, password_reset_tokens, dan sessions didefinisikan pada satu file migration bawaan Laravel. Kolom password bertipe string, dan kolom email diberi indeks unik sehingga satu email hanya boleh dipakai satu akun.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

3.5. View Authentication

View register menyediakan empat kolom input, yaitu nama, email, password, dan konfirmasi password. Form dikirim ke route register memakai method POST dan setiap form menyertakan directive @csrf sebagai token keamanan.

```blade
<x-guest-layout>
    <form method="POST" action="{{ route('register') }}">
        @csrf

        <!-- Name -->
        <div>
            <x-input-label for="name" :value="__('Name')" />
            <x-text-input id="name" class="block mt-1 w-full" type="text" name="name" :value="old('name')" required autofocus autocomplete="name" />
            <x-input-error :messages="$errors->get('name')" class="mt-2" />
        </div>

        <!-- Email Address -->
        <div class="mt-4">
            <x-input-label for="email" :value="__('Email')" />
            <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <!-- Password -->
        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')" />

            <x-text-input id="password" class="block mt-1 w-full"
                          type="password"
                          name="password"
                          required autocomplete="new-password" />

            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <!-- Confirm Password -->
        <div class="mt-4">
            <x-input-label for="password_confirmation" :value="__('Confirm Password')" />

            <x-text-input id="password_confirmation" class="block mt-1 w-full"
                          type="password"
                          name="password_confirmation" required autocomplete="new-password" />

            <x-input-error :messages="$errors->get('password_confirmation')" class="mt-2" />
        </div>

        <div class="flex items-center justify-end mt-4">
            <a class="underline text-sm text-gray-600 hover:text-gray-900 rounded-md focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500" href="{{ route('login') }}">
                {{ __('Already registered?') }}
            </a>

            <x-primary-button class="ms-4">
                {{ __('Register') }}
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

View login berisi kolom email, password, dan checkbox Remember me, dengan tautan lupa password di bawahnya. Pesan kesalahan ditampilkan melalui komponen x-input-error yang membaca daftar error dari controller.

```
<x-guest-layout>
    <!-- Session Status -->
    <x-auth-session-status class="mb-4" :status="session('status')" />

    <form method="POST" action="{{ route('login') }}">
        @csrf

        <!-- Email Address -->
        <div>
            <x-input-label for="email" :value="__('Email')" />
            <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autofocus autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <!-- Password -->
        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')" />

            <x-text-input id="password" class="block mt-1 w-full"
                            type="password"
                            name="password"
                            required autocomplete="current-password" />

            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <!-- Remember Me -->
        <div class="block mt-4">
            <label for="remember_me" class="inline-flex items-center">
                <input id="remember_me" type="checkbox" class="rounded border-gray-300 text-indigo-600 shadow-sm focus:ring-indigo-500" name="remember">
                <span class="ms-2 text-sm text-gray-600">{{ __('Remember me') }}</span>
            </label>
        </div>

        <div class="flex items-center justify-end mt-4">
            @if (Route::has('password.request'))
                <a class="underline text-sm text-gray-600 hover:text-gray-900 rounded-md focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500" href="{{ route('password.request') }}">
                    {{ __('Forgot your password?') }}
                </a>
            @endif

            <x-primary-button class="ms-3">
                {{ __('Log in') }}
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

Halaman dashboard masih sama seperti bawaan Breeze. Isinya hanya kartu sederhana yang menampilkan teks bahwa pengguna sudah login, dan header halaman memakai komponen x-app-layout.

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    {{ __("You're logged in!") }}
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

3.6. Menu Pengguna dan Tombol Logout

Menu Profile dan Log Out berada pada layouts/navigation.blade.php. Nama pengguna diambil dari Auth::user()->name, dan tombol Log Out dikirim sebagai form POST ke route logout karena proses logout tidak boleh dilakukan melalui permintaan GET. Potongan kode menu tersebut adalah sebagai berikut, sedangkan file lengkapnya juga memuat versi menu untuk layar kecil.

```blade
<!-- Potongan menu pengguna pada layouts/navigation.blade.php -->
                            <div class="ms-1">
                                <svg class="fill-current h-4 w-4" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 20 20">
                                    <path fill-rule="evenodd" d="M5.293 7.293a1 1 0 011.414 0L10 10.586l3.293-3.293a1 1 0 111.414 1.414l-4 4a1 1 0 01-1.414 0l-4-4a1 1 0 010-1.414z" clip-rule="evenodd" />
                                </svg>
                            </div>
                        </button>
                    </x-slot>

                    <x-slot name="content">
                        <x-dropdown-link :href="route('profile.edit')">
                            {{ __('Profile') }}
                        </x-dropdown-link>

                        <!-- Authentication -->
                        <form method="POST" action="{{ route('logout') }}">
                            @csrf

                            <x-dropdown-link :href="route('logout')"
                                    onclick="event.preventDefault();
                                                this.closest('form').submit();">
                                {{ __('Log Out') }}
                            </x-dropdown-link>
                        </form>
                    </x-slot>
                </x-dropdown>
            </div>

            <!-- Hamburger -->
            <div class="-me-2 flex items-center sm:hidden">
                <button @click="open = ! open" class="inline-flex items-center justify-center p-2 rounded-md text-gray-400 hover:text-gray-500 hover:bg-gray-100 focus:outline-none focus:bg-gray-100 focus:text-gray-500 transition duration-150 ease-in-out">
                    <svg class="h-6 w-6" stroke="currentColor" fill="none" viewBox="0 0 24 24">
                        <path :class="{'hidden': open, 'inline-flex': ! open }" class="inline-flex" stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 6h16M4 12h16M4 18h16" />
                        <path :class="{'hidden': ! open, 'inline-flex': open }" class="hidden" stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M6 18L18 6M6 6l12 12" />
                    </svg>
                </button>
            </div>
        </div>
```

3.7. Membuat User Dummy dengan Seeder dan Faker

Agar pengujian login tidak selalu memakai akun hasil register, saya membuat seeder untuk mengisi tabel users secara otomatis. File database/seeders/UserSeeder.php diisi dengan perulangan lima kali untuk membuat akun dummy dan satu akun admin manual yang mudah diingat.

```php
<?php

namespace Database\Seeders;


use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Faker\Factory as Faker;


class UserSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $faker = Faker::create();

        for($i = 0; $i < 5; $i++){
            DB::table('users')->insert([
                'name' => $faker->name,
                'email' => $faker->unique()->safeEmail,
                'password' => Hash::make('password'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }

        DB::table('users') -> insert([
            'name' => 'Admin Demo',
            'email' => 'admin@gmail.com',
            'password' => Hash::make('admin123'),
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    }
}
```

- Faker::create() membuat objek Faker. Karena locale tidak ditentukan, nama yang dihasilkan berbahasa Inggris, misalnya Omari Sawayn V yang terlihat pada Gambar 4.12.
- $faker->unique()->safeEmail menghasilkan email acak yang dijamin belum pernah dipakai, sehingga tidak terjadi galat duplicate entry pada kolom email yang bersifat unik.

- Hash::make('password') membuat seluruh akun dummy memakai password yang sama, tetapi tetap tersimpan dalam bentuk hash.
- Perulangan for ($i = 0; $i < 5; $i++) dan DB::table('users')->insert([...]) menghasilkan 5 baris data dummy melalui Query Builder.
- Akun admin manual pada project ini memakai email admin@gmail.com dengan password admin123, berbeda dari teks modul yang menuliskan admin@example.com. Password akun admin juga disimpan dalam bentuk hash.

Seeder dijalankan dengan perintah php artisan db:seed --class=UserSeeder seperti berikut.

```
> php artisan db:seed --class=UserSeeder

  INFO  Seeding database.

  Database\Seeders\UserSeeder ............................... RUNNING
  Database\Seeders\UserSeeder ............................ 963 ms DONE
```

Catatan: seeder ini belum sepenuhnya idempotent. Jika dijalankan dua kali tanpa membersihkan tabel users lebih dulu, proses akan berhenti pada baris akun admin karena email admin@gmail.com sudah ada di database.

3.8. Pengujian Route dan Middleware Auth

Sebelum aplikasi diuji di browser, saya memeriksa daftar route yang terdaftar dengan php artisan route:list. Perintah ini memperlihatkan pasangan route register, login, logout, dan profile beserta controller yang menanganinya.

```
> php artisan route:list --except-vendor

  GET|HEAD  / ........................................... routes/web.php:6
  GET|HEAD  confirm-password ........ password.confirm >
Auth\ConfirmablePasswordController@show
  POST      confirm-password ................
Auth\ConfirmablePasswordController@store
  GET|HEAD  dashboard ................................ dashboard >
routes/web.php:10
  GET|HEAD  forgot-password ......... password.request >
Auth\PasswordResetLinkController@create
  POST      forgot-password ............ password.email >
Auth\PasswordResetLinkController@store
  GET|HEAD  login ........................ login >
Auth\AuthenticatedSessionController@create
  POST      login ................................
Auth\AuthenticatedSessionController@store
  POST      logout ................................ logout >
Auth\AuthenticatedSessionController@destroy
  PUT       password ................................. password.update >
Auth\PasswordController@update
  GET|HEAD  profile ........................................ profile.edit >
ProfileController@edit
  PATCH     profile ........................................ profile.update
> ProfileController@update
  DELETE    profile ..................................... profile.destroy >
ProfileController@destroy
```

```
  GET|HEAD  register ..........................  register >
Auth\RegisteredUserController@create
  POST      register ....................................
Auth\RegisteredUserController@store

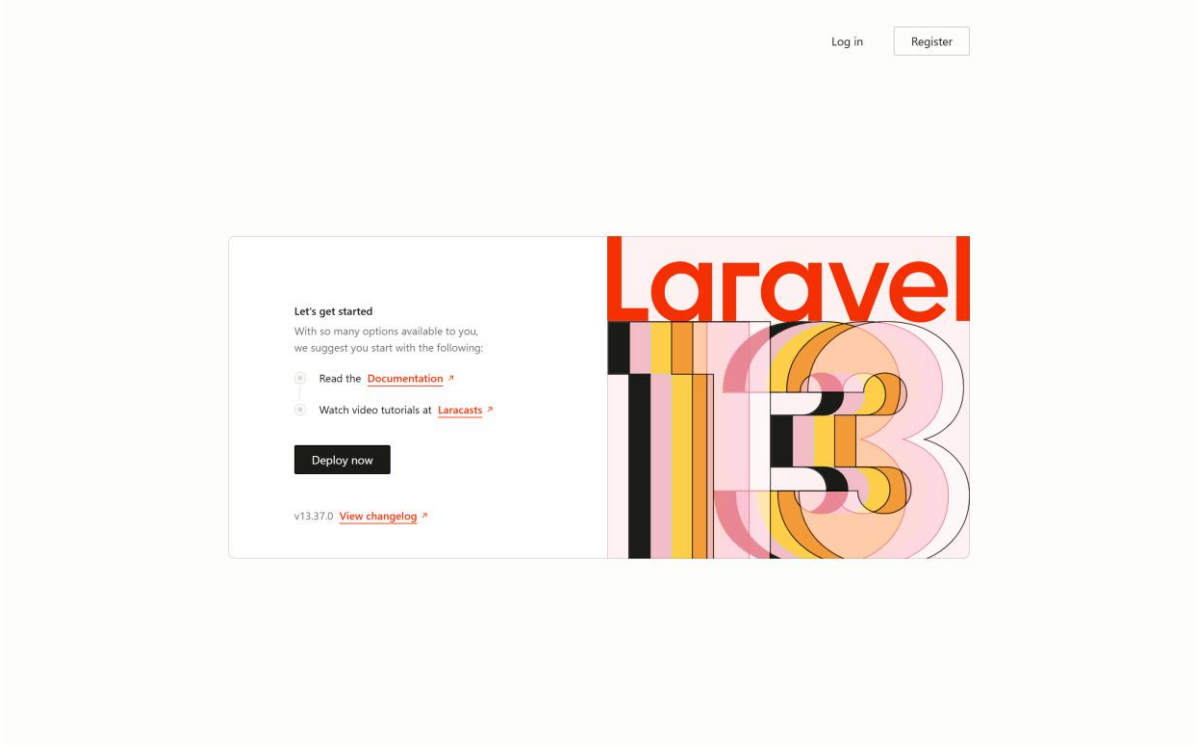
                                                                  Showing
[20] routes
```

Saat alamat http://127.0.0.1:8000/dashboard dibuka tanpa login, middleware auth langsung mengalihkan permintaan ke halaman login. Setelah login, halaman yang sama dapat dibuka dan menampilkan nama pengguna pada menu di kanan atas, seperti yang terdokumentasi pada bagian berikutnya. Perlu dicatat bahwa route dashboard memakai middleware verified, tetapi model User tidak memakai kontrak MustVerifyEmail, sehingga pengguna yang baru mendaftar tetap bisa masuk tanpa verifikasi email.

4. Hasil dan Dokumentasi Program

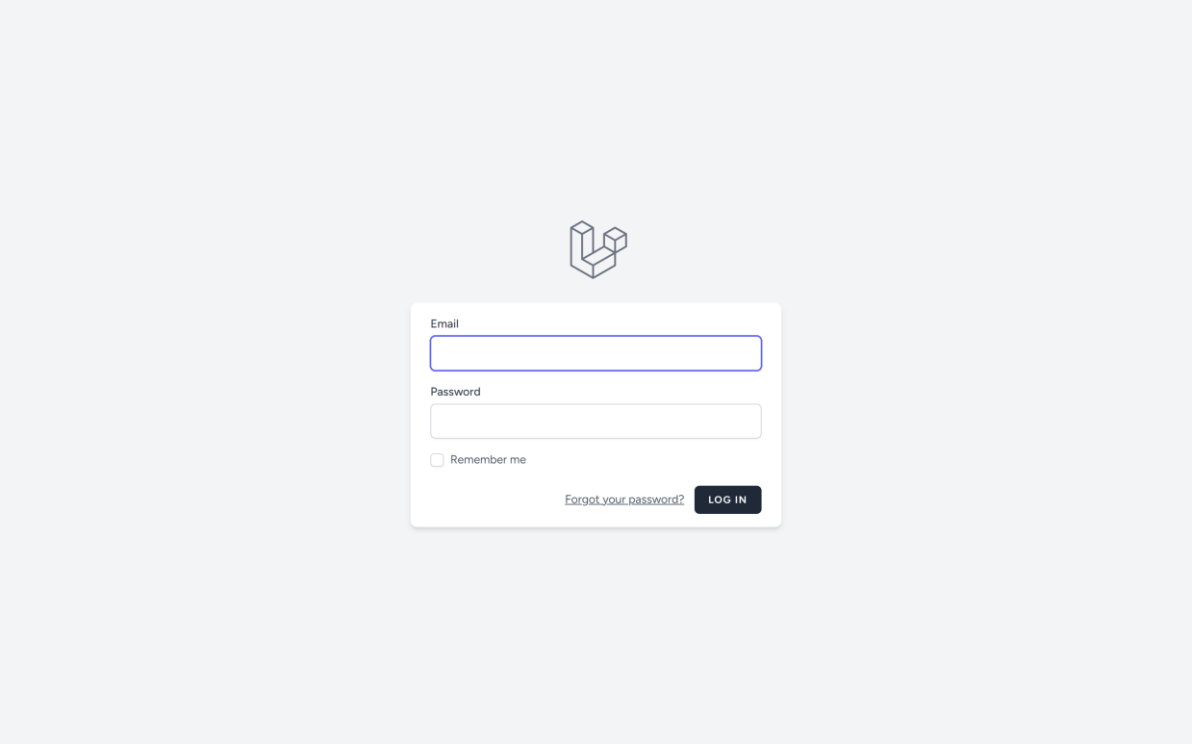
Setelah migration dan seeder dijalankan serta server dinyalakan dengan php artisan serve, aplikasi diuji melalui browser Google Chrome. Dokumentasi di bawah ini diambil dari aplikasi yang benar-benar berjalan pada alamat http://127.0.0.1:8000 dan dari phpMyAdmin untuk melihat isi database.

Gambar 4.1. Tampilan Halaman Welcome



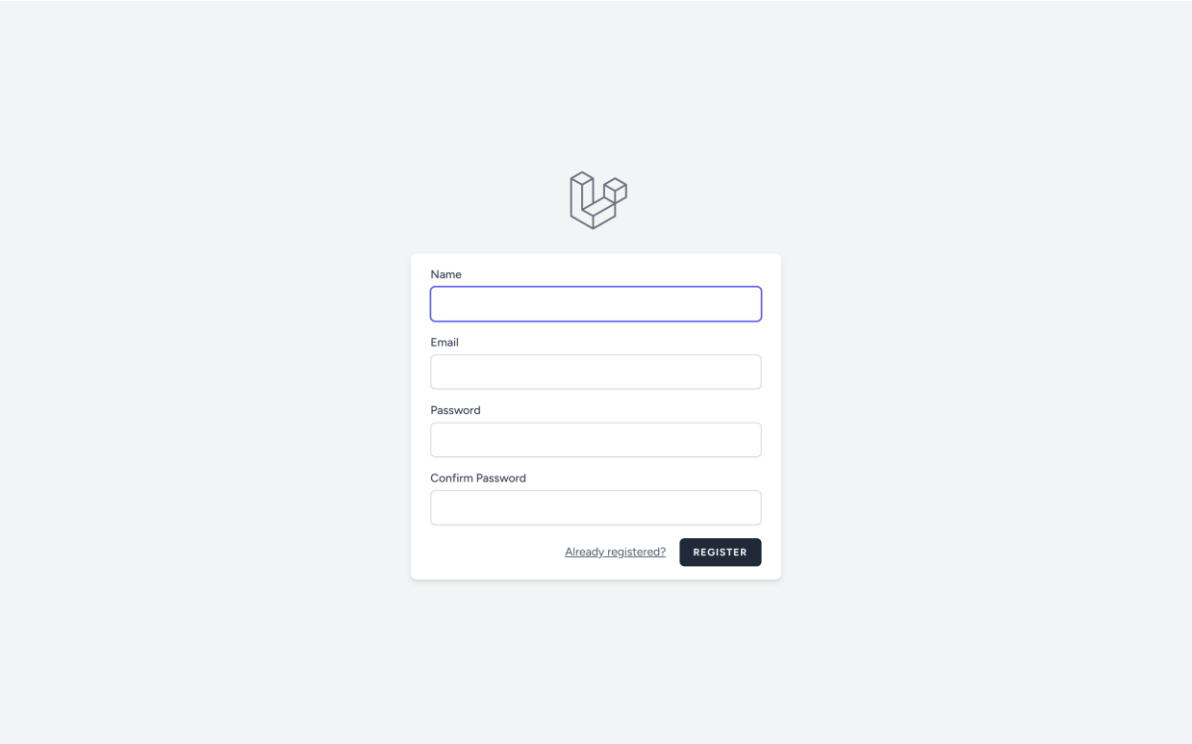
Keterangan Gambar 4.1: Halaman awal aplikasi sebelum login. Di pojok kanan atas tersedia tautan Log in dan tombol Register yang mengarah ke route login dan register dari Breeze.

Gambar 4.2. Halaman Dashboard Dialihkan ke Login



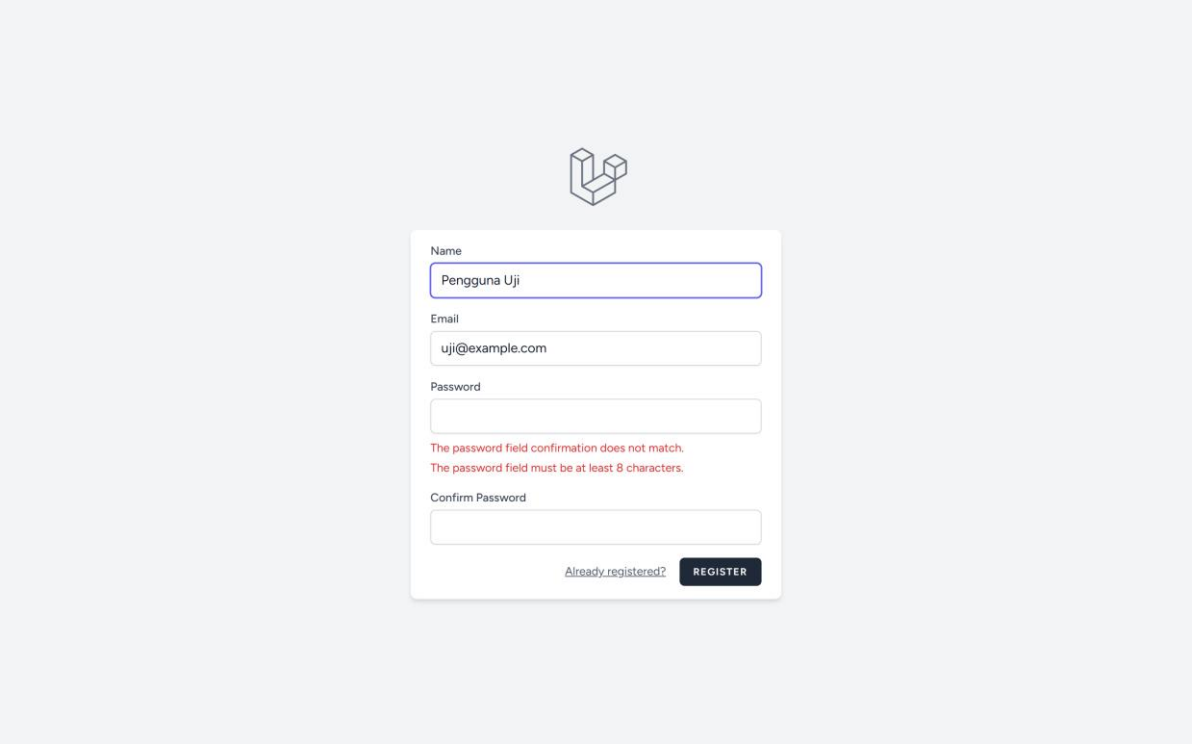
Keterangan Gambar 4.2: Hasil percobaan membuka http://127.0.0.1:8000/dashboard tanpa login. Middleware auth menolak permintaan dan mengalihkan browser ke /login, sehingga isi dashboard tidak pernah tampil.

Gambar 4.3. Tampilan Halaman Register



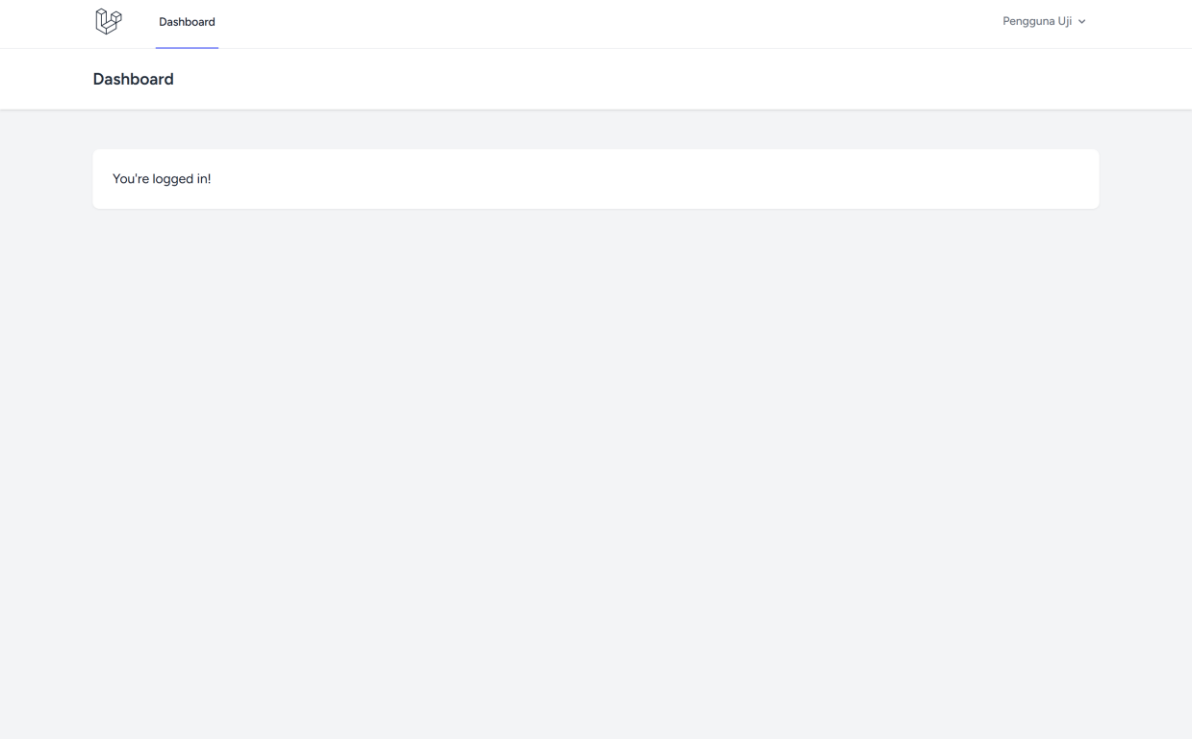
Keterangan Gambar 4.3: Form pendaftaran dari resources/views/auth/register.blade.php dengan kolom Name, Email, Password, dan Confirm Password.

Gambar 4.4. Pesan Validasi pada Halaman Register



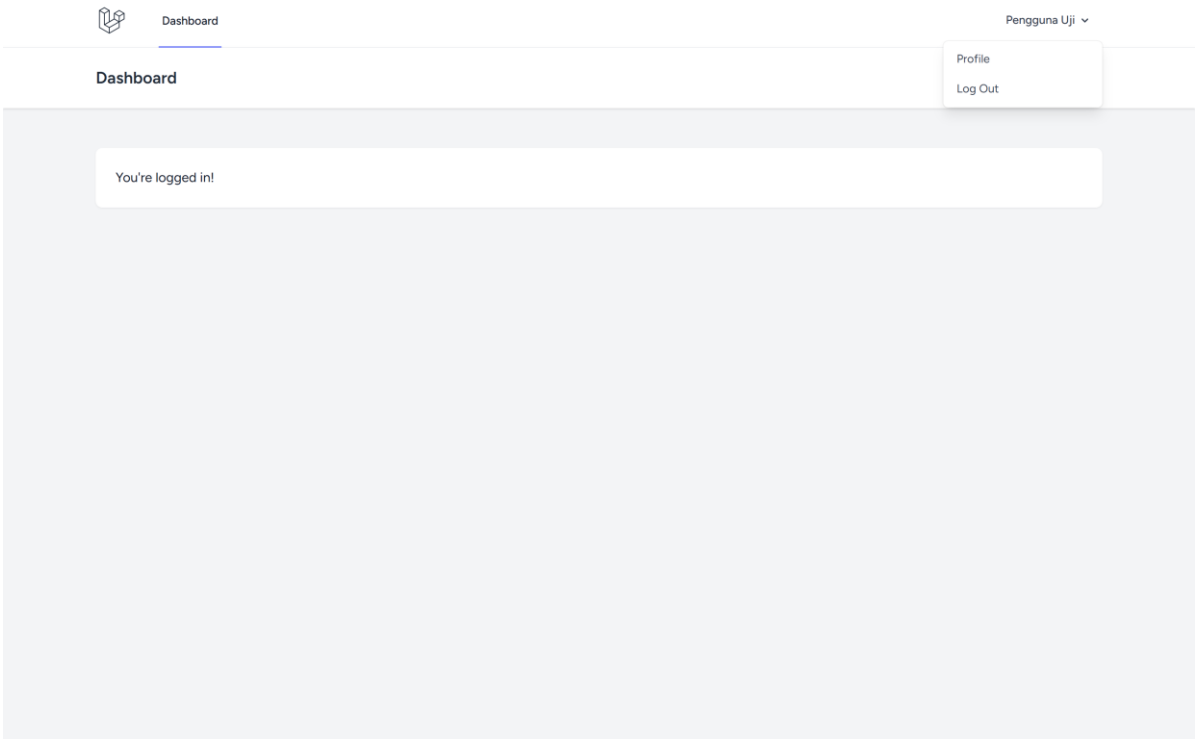
Keterangan Gambar 4.4: Percobaan mendaftar dengan password kurang dari 8 karakter dan konfirmasi yang tidak sama. Controller menolak data tersebut dan menampilkan pesan "The password field must be at least 8 characters" serta "The password field confirmation does not match" di bawah kolom password.

Gambar 4.5. Dashboard Setelah Berhasil Mendaftar



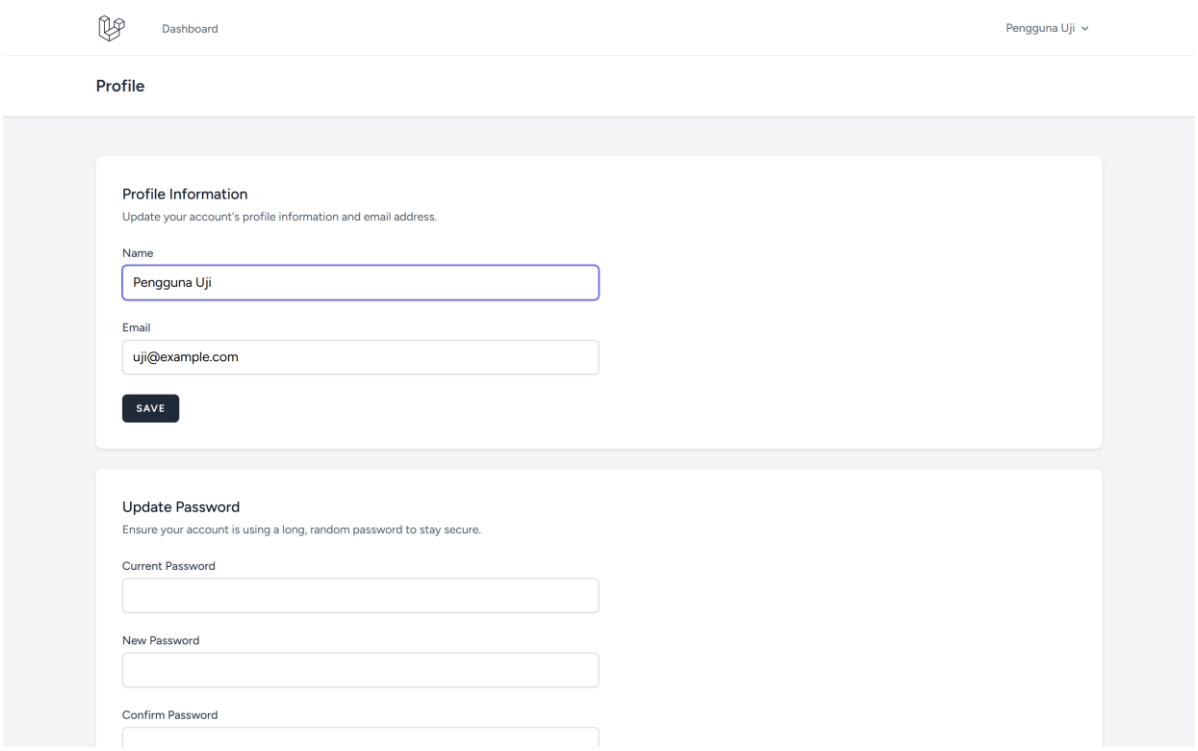
Keterangan Gambar 4.5: Setelah data register yang valid dikirim, akun baru tersimpan dan pengguna otomatis login. Nama pengguna yang baru didaftarkan tampil pada menu kanan atas dan halaman dashboard dapat dibuka.

Gambar 4.6. Menu Pengguna pada Halaman Dashboard



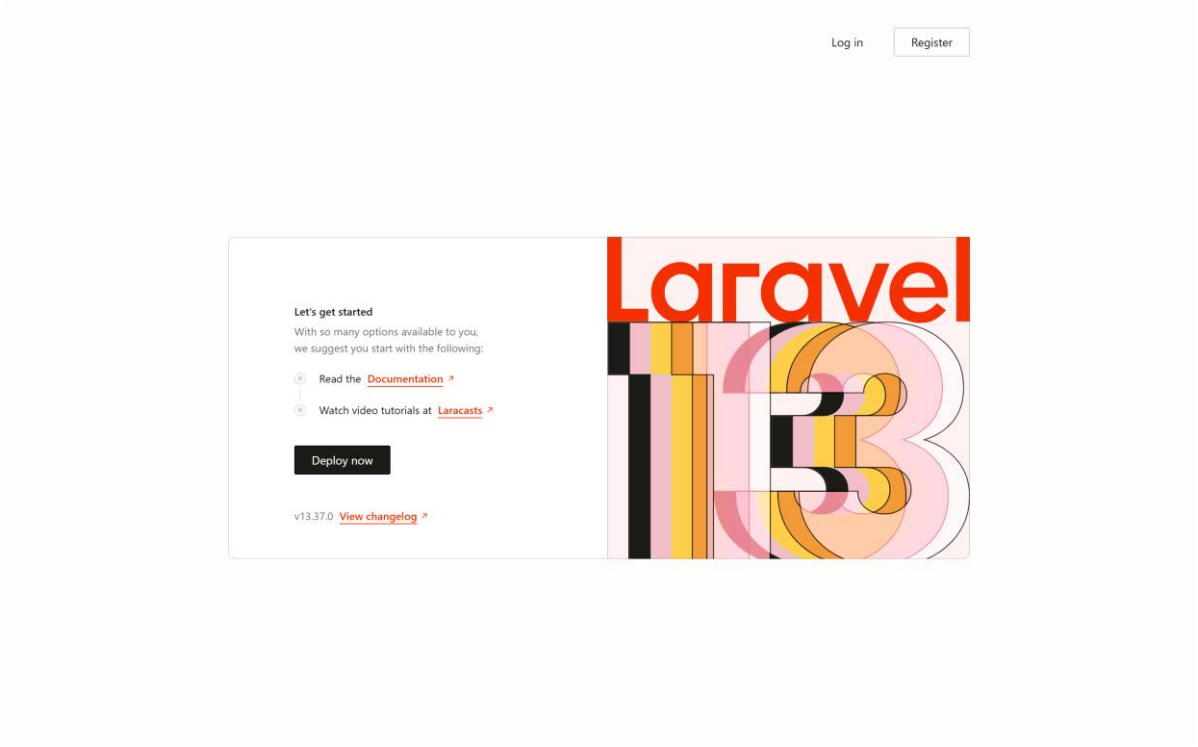
Keterangan Gambar 4.6: Menu dropdown pada layouts/navigation.blade.php berisi pilihan Profile dan Log Out. Kedua pilihan inilah yang menghubungkan pengguna ke halaman profil dan mengakhiri session login.

Gambar 4.7. Tampilan Halaman Profile



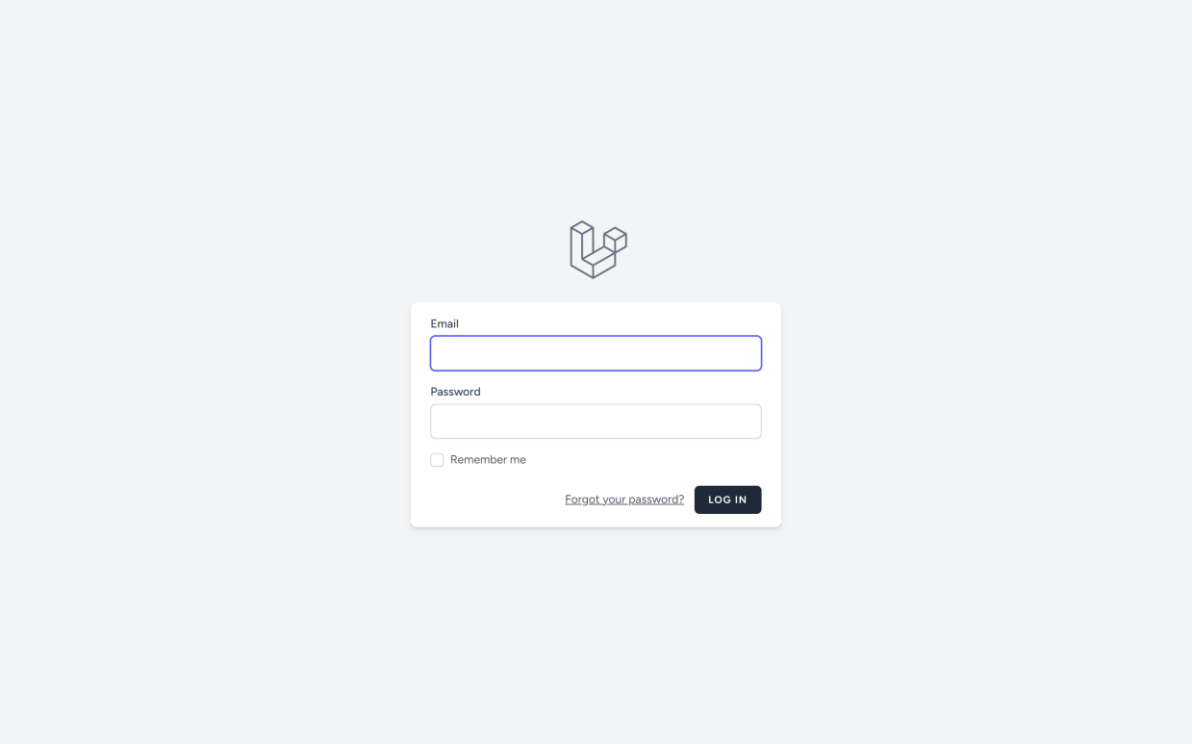
Keterangan Gambar 4.7: Halaman profile bawaan Breeze berisi form perubahan nama dan email, form penggantian password, serta form penghapusan akun. Halaman ini hanya bisa dibuka setelah login karena route-nya memakai middleware auth.

Gambar 4.8. Halaman Welcome Setelah Logout



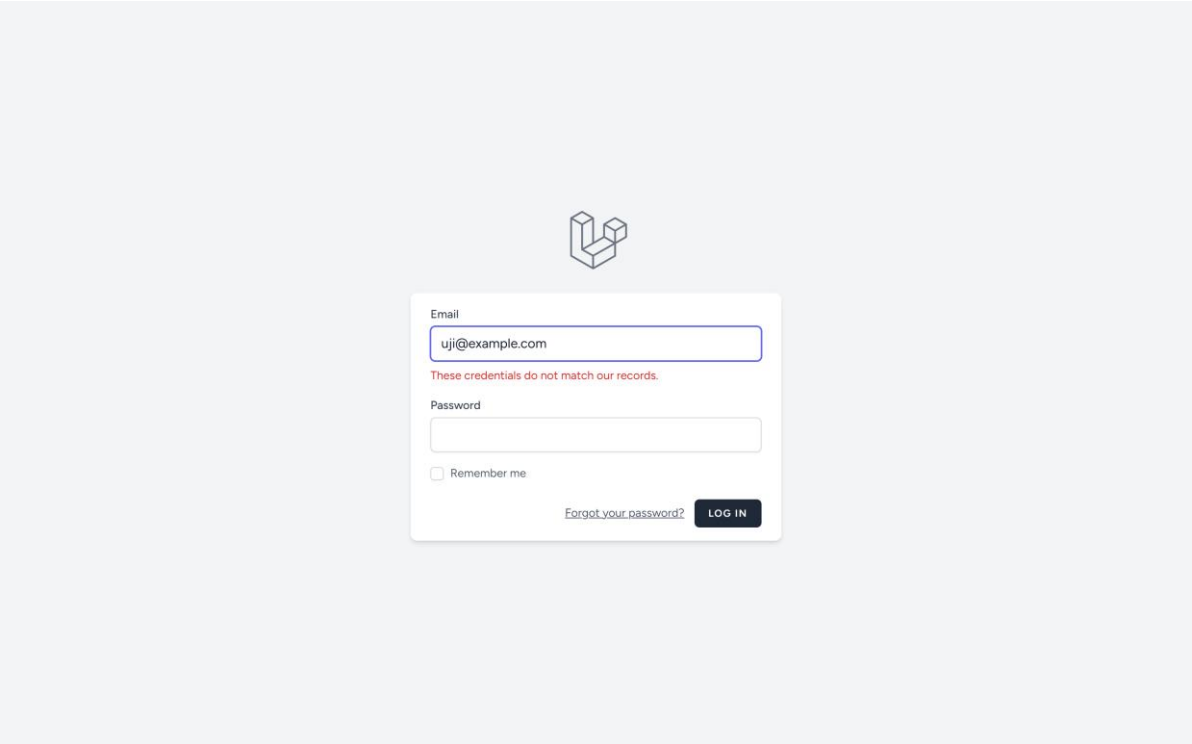
Keterangan Gambar 4.8: Setelah Log Out ditekan, session dihapus dan pengguna diarahkan kembali ke halaman utama. Menu di pojok kanan atas berubah kembali menjadi Log in dan Register.

Gambar 4.9. Tampilan Halaman Login



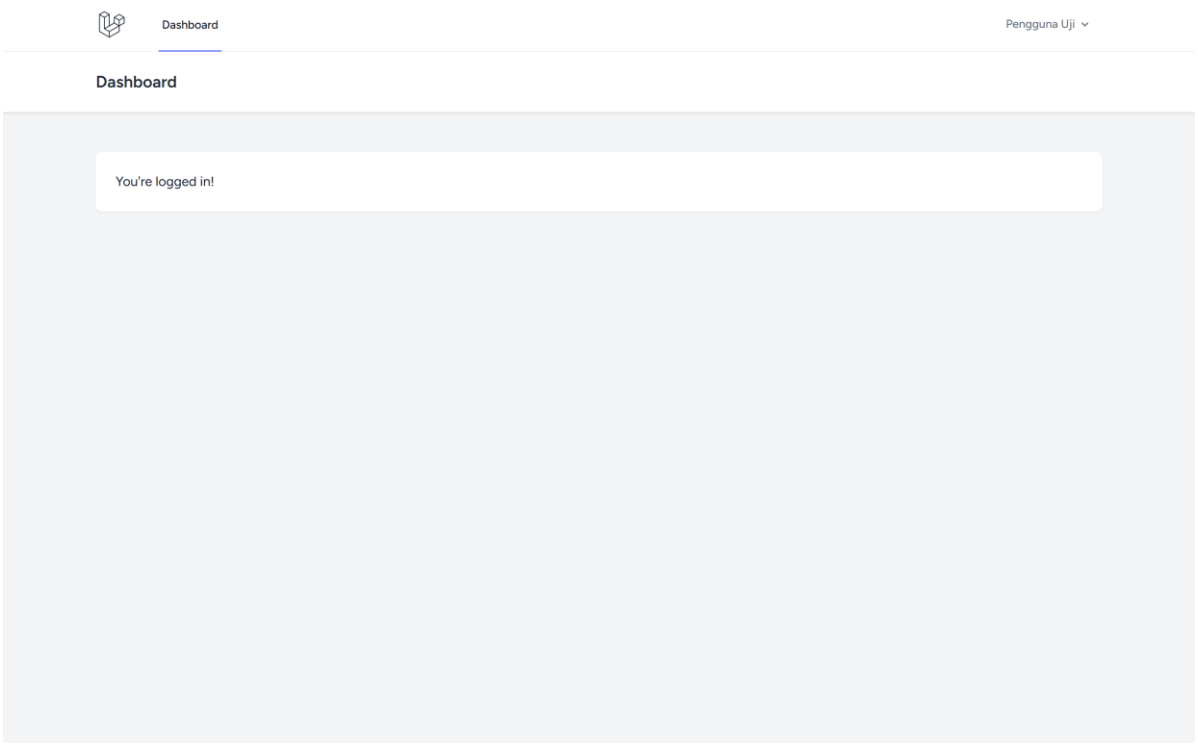
Keterangan Gambar 4.9: Form login dari resources/views/auth/login.blade.php dengan kolom Email, Password, checkbox Remember me, dan tautan lupa password.

Gambar 4.10. Pesan Galat Saat Password Salah



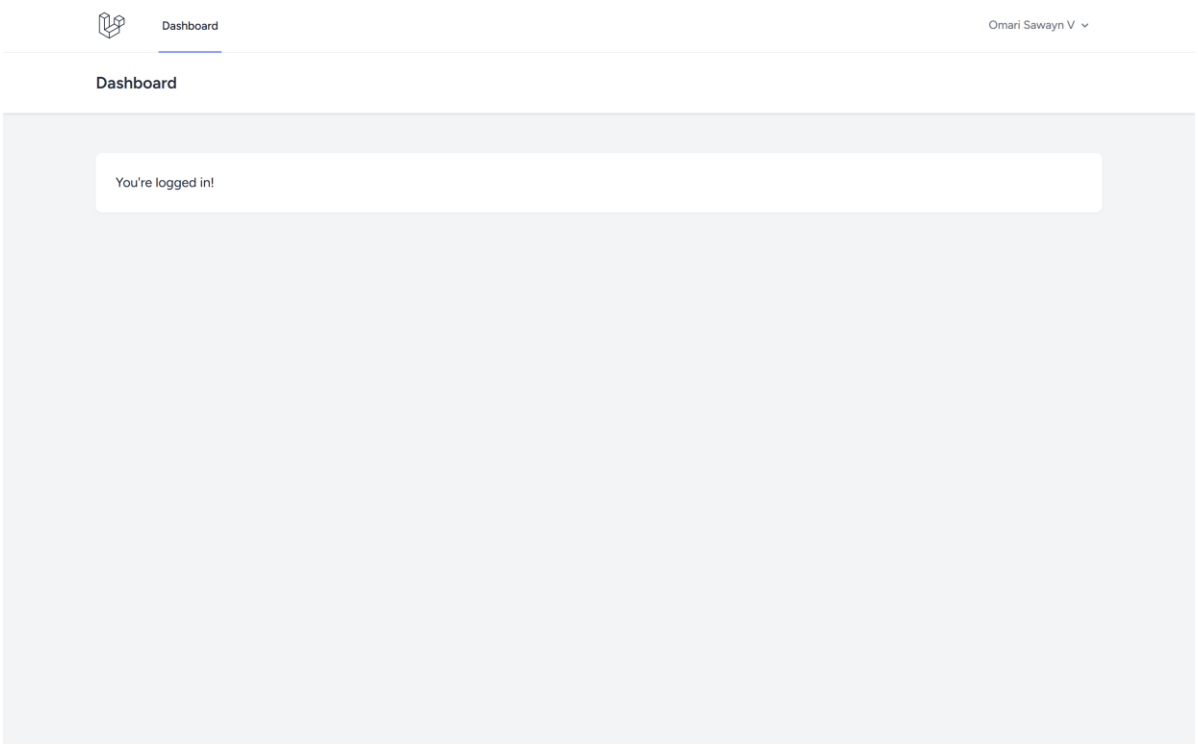
Keterangan Gambar 4.10: Percobaan login memakai password yang salah. LoginRequest mengembalikan pesan "These credentials do not match our records." dan permintaan gagal tersebut ikut dihitung oleh RateLimiter.

Gambar 4.11. Dashboard Setelah Login yang Berhasil



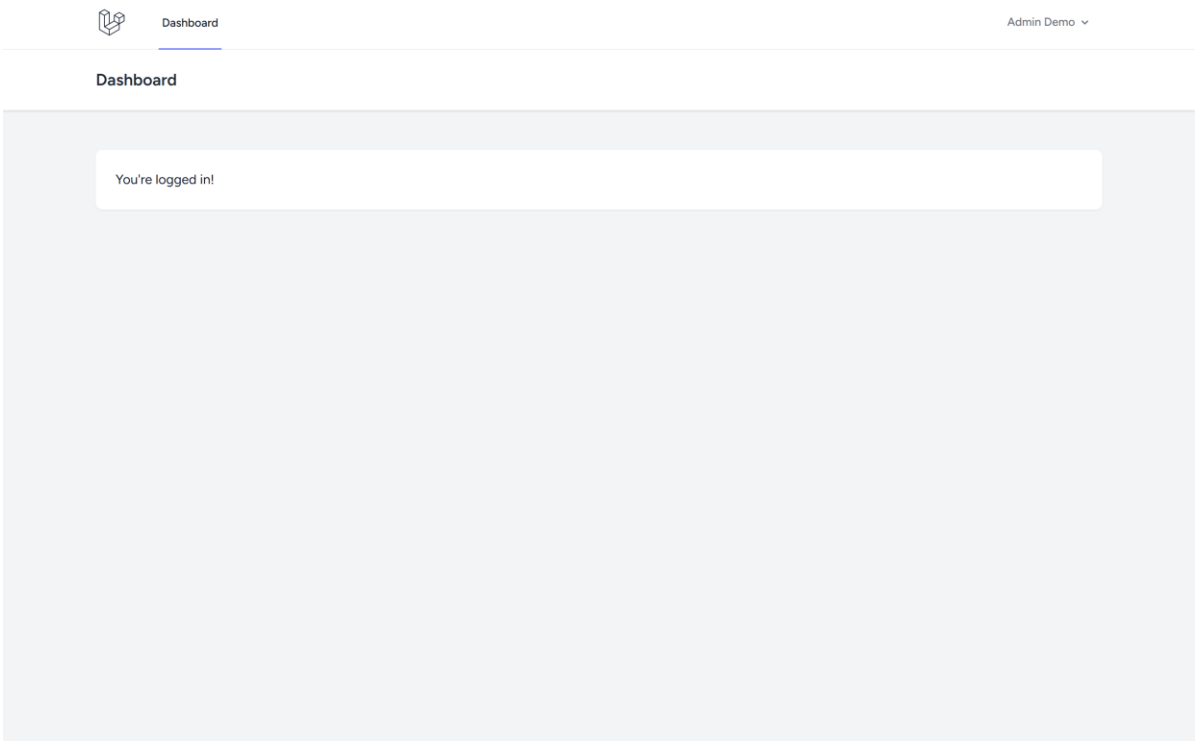
Keterangan Gambar 4.11: Login memakai email dan password yang benar mengarahkan pengguna ke dashboard, dan session login tersimpan di tabel sessions.

Gambar 4.12. Login Memakai Akun Dummy Hasil Seeder



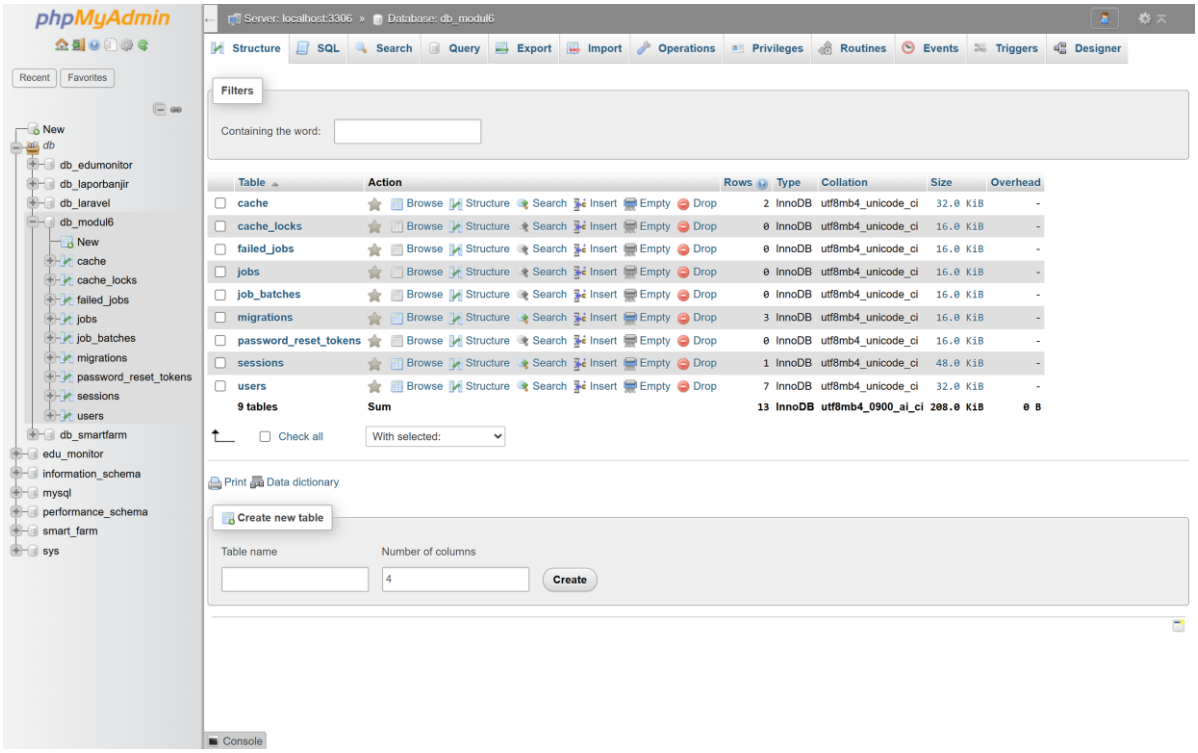
Keterangan Gambar 4.12: Akun dummy hasil seeder dapat dipakai untuk login dengan password "password". Nama pengguna pada menu kanan atas mengikuti nama acak yang dibuat Faker, yang pada pengujian ini adalah Omari Sawayn V.

Gambar 4.13. Login Memakai Akun Admin Manual



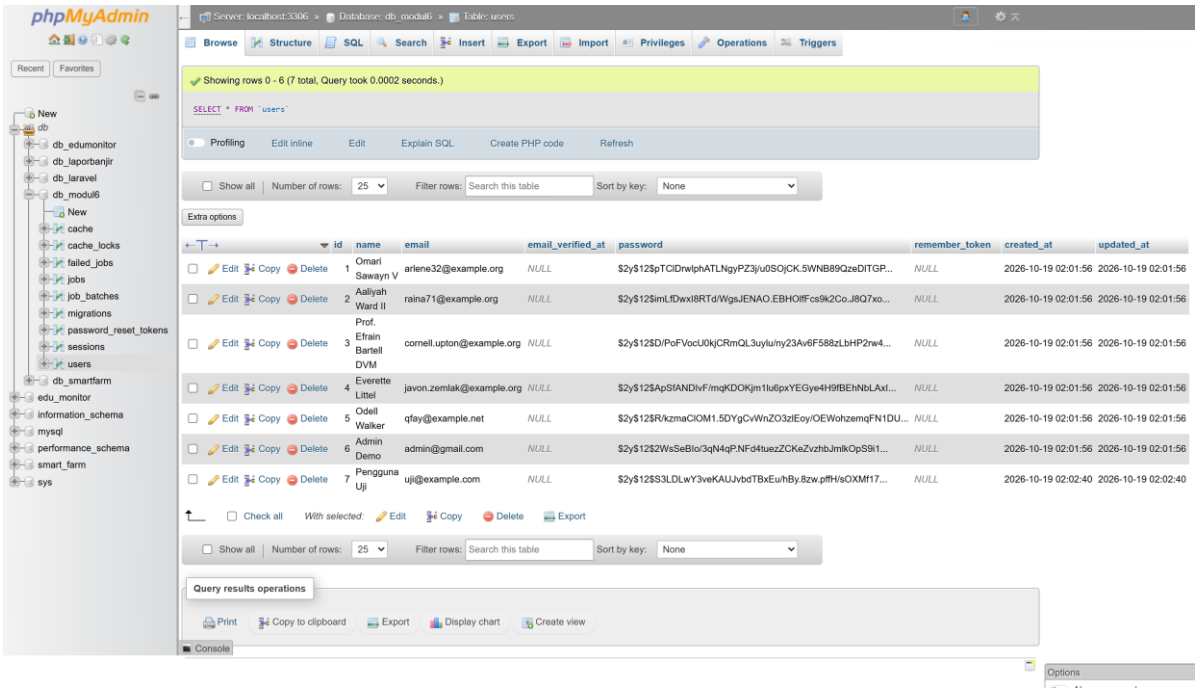
Keterangan Gambar 4.13: Akun admin manual pada UserSeeder dengan email admin@gmail.com dan password admin123 juga berhasil login tanpa perlakuan khusus, sama seperti akun hasil register.

Gambar 4.14. Struktur Database db_modul6 pada phpMyAdmin



Keterangan Gambar 4.14: Database db_modul6 berisi 9 tabel, yaitu tabel bawaan Laravel untuk authentication (users, password_reset_tokens, sessions), tabel pendukung (cache, cache_locks, jobs, job_batches, failed_jobs), dan tabel migrations. Tabel users berisi 7 baris setelah pengujian register.

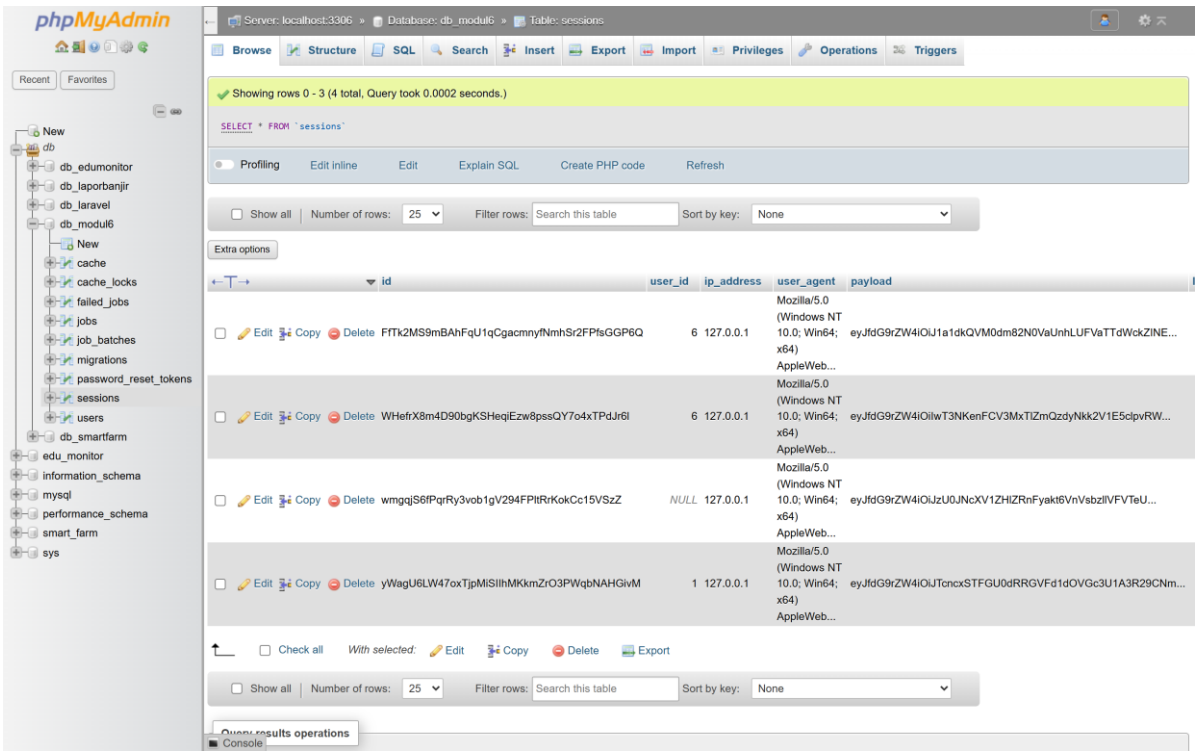
Gambar 4.15. Isi Tabel users pada phpMyAdmin



Keterangan Gambar 4.15: Tabel users berisi 5 akun dummy hasil seeder, 1 akun admin manual, dan 1 akun hasil register. Kolom password seluruhnya berisi deretan karakter acak

yang diawali $2y$12$, yang merupakan hasil hashing algoritma bcrypt, bukan password asli yang diketik.

Gambar 4.16. Isi Tabel sessions pada phpMyAdmin



Keterangan Gambar 4.16: SESSION_DRIVER pada project ini disetel database, sehingga setiap login meninggalkan satu baris pada tabel sessions. Baris dengan nilai user_id terisi menandakan session milik pengguna yang sedang login, sedangkan baris dengan user_id NULL berasal dari pengunjung yang belum login.

Penjelasan hasil tampilan:

- Halaman dashboard tidak dapat dibuka sebelum login. Percobaan pada Gambar 4.2 berakhir di halaman login, yang berarti middleware auth bekerja sebelum request diteruskan ke view.
- Data akun pada Gambar 4.15 tidak diisi manual semuanya. Lima baris pertama berasal dari UserSeeder melalui Faker, satu baris dari akun admin manual, dan satu baris dari proses register pada halaman web.
- Kolom password pada tabel users menyimpan hasil hashing. Nilai seperti $2y$12$pTClD... tidak dapat dibaca kembali menjadi password asli, sehingga isi database tetap aman walau dilihat orang lain.
- Tabel sessions membuktikan bahwa status login disimpan di sisi server. Laravel mengenali pengguna yang sudah login dari baris session yang cocok dengan cookie pada browser.
- Akun hasil seeder (Gambar 4.12) dan akun hasil register (Gambar 4.11) dapat login dengan cara yang sama, karena keduanya sama-sama tersimpan pada tabel users dengan password yang sudah di-hash.

Berkas-berkas yang dihasilkan pada praktikum ini adalah .env, routes/web.php, routes/auth.php, app/Http/Controllers/Auth/RegisteredUserController.php, app/Http/Controllers/Auth/AuthenticatedSessionController.php, app/Http/Requests/Auth/LoginRequest.php, app/Http/Controllers/ProfileController.php, app/Models/User.php, database/migrations/0001_01_01_000000_create_users_table.php, database/seeders/UserSeeder.php, resources/views/auth/register.blade.php, resources/views/auth/login.blade.php, resources/views/dashboard.blade.php, dan resources/views/layouts/navigation.blade.php.

5. Analisis dan Pembahasan

Secara umum praktikum ini berjalan sesuai alur yang dijelaskan modul. Fitur register, login, dan logout dari Breeze dapat dipakai tanpa perlu menulis route maupun controller authentication dari awal, dan bagian yang perlu penyesuaian hanya konfigurasi database serta penambahan seeder untuk akun dummy.

- Pembagian tugas antar file cukup jelas. Route hanya memetakan alamat URL ke controller, controller menangani validasi dan penyimpanan data, view menampilkan form, dan model User menjadi perantara tabel users. Pola MVC yang sudah dipakai pada modul sebelumnya tetap terlihat pada fitur authentication ini.
- Session dipakai untuk mengingat pengguna yang sudah login. Setelah Auth::attempt() berhasil, Laravel menyimpan identitas pengguna pada session dan meregenerasi ID session-nya. Karena SESSION_DRIVER disetel database, jejaknya terlihat pada tabel sessions di phpMyAdmin.
- Password tidak pernah tersimpan dalam bentuk asli. Hash::make() dipakai saat register dan Hash::make('password') dipakai pada seeder, sedangkan pencocokan saat login dilakukan Hash::check() di dalam Auth::attempt(). Semua nilai pada kolom password diawali $2y$, yang menandakan algoritma bcrypt.
- Middleware auth menjadi penjaga halaman. Route dashboard memakai middleware auth dan verified, sedangkan route profile memakai middleware auth. Uji coba pada Gambar 4.2 memperlihatkan bahwa permintaan ke halaman terlindungi tanpa session berakhir pada pengalihan ke /login.
- Faker mempermudah pembuatan akun uji. Cukup mengubah jumlah perulangan pada UserSeeder untuk menambah akun dummy, dan kata unique() mencegah email kembar yang akan memicu galat duplicate entry. Karena Faker::create() dipanggil tanpa locale, nama yang dihasilkan berbahasa Inggris.
- Seeder ini belum idempotent. Menjalankannya dua kali pada tabel yang sama akan gagal pada baris akun admin karena email admin@gmail.com sudah terdaftar, sehingga sebelum menjalankan ulang seeder, tabel users perlu dibersihkan lebih dulu melalui php artisan migrate:fresh.

Kendala dan Penyelesaian: beberapa hal yang saya temui selama praktikum adalah sebagai berikut.

- Perintah npm tidak dikenali pada PowerShell dan muncul pesan bahwa npm.ps1 cannot be loaded because running scripts is disabled on this system. Persoalan ini terjadi karena

kebijakan eksekusi script PowerShell. Saya menjalankan perintah npm melalui Command Prompt, dan cara lain yang sejalan dengan modul 6.4.9 adalah menjalankan Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass pada terminal yang dipakai.

- Vite tidak menemukan port kosong. Saat npm run dev dijalankan, port 5173 sudah dipakai proses lain sehingga Vite otomatis pindah ke port 5174 dan menuliskan alamat barunya ke public/hot. Aplikasi tetap berjalan karena Laravel membaca file hot tersebut, tetapi terminal kedua tidak boleh ditutup selama pengujian.
- phpMyAdmin bawaan Laragon di alamat http://localhost/phpmyadmin/ gagal dibuka dengan pesan require_once(.../libraries/constants.php): Failed to open stream: No such file or directory. Sebagai gantinya saya memakai phpMyAdmin 6.0.0 pada alamat http://localhost/phpmyadmin/public/ yang masih berfungsi normal.
- Teks modul menuliskan akun admin manual dengan email admin@example.com, sedangkan pada project ini seeder memakai admin@gmail.com. Perbedaan ini disengaja agar akun pengujian memakai domain yang umum dipakai, dan tidak berpengaruh terhadap proses login.

6. Kesimpulan

Praktikum ini menunjukkan bahwa fitur authentication pada Laravel dapat dibangun dengan cepat memakai Laravel Breeze, tetapi proses di baliknya tetap dapat dipelajari dari file yang dihasilkan. Register berjalan dengan memvalidasi input, menyimpan password hasil Hash::make(), dan langsung membuat session login. Login mencocokkan email dan password melalui LoginRequest sebelum Laravel menyimpan session untuk pengguna tersebut. Middleware auth memeriksa session sebelum halaman terlindungi dibuka, dan logout menghapus session tersebut sehingga halaman yang sama tidak dapat diakses lagi.

Penambahan seeder dengan Faker membuat pengujian login lebih praktis karena akun dummy dapat dibuat sekaligus dalam jumlah banyak, dengan syarat kolom password tetap diisi hash dan email yang dihasilkan bersifat unik. Seluruh hasil pengujian pada browser dan pemeriksaan isi database melalui phpMyAdmin memperlihatkan bahwa data akun tersimpan pada tabel users dengan password yang sudah di-hash, dan status login tersimpan pada tabel sessions.

7. Lampiran

Tautan Repositori GitHub:

https://github.com/Afza-code/PABW-semester-3/tree/main/modul6